

Gerenciamento de Processos

Objetivo

Explorar o gerenciamento de processos em um sistema Linux, observando como o sistema operacional aloca recursos, cria e encerra processos e monitora CPU e memória em tempo real.

Pré-requisitos

- Docker instalado na máquina
- Docker Compose instalado

Como iniciar o laboratório

Suba o container com o comando abaixo. O **-d** significa que ele vai rodar em segundo plano (detached), sem travar o terminal:

```
docker compose up -d
```

Agora entre dentro do container para começar os exercícios. É como "entrar" em outra máquina Linux:

```
docker compose exec so-lab-01 bash --login
```

Exercícios

Exercício 1 — Listando processos

O comando **ps aux** mostra todos os processos rodando no sistema. As colunas mais importantes são:

- **PID** — identificador único do processo
- **%CPU** — quanto de processador ele está usando
- **%MEM** — quanto de memória ele está usando
- **COMMAND** — qual programa está rodando

```
ps aux
```

Para listar os 10 processos que mais consomem CPU:

```
ps aux --sort=-%cpu | head -10
```

--sort=-%cpu ordena pela coluna CPU em ordem decrescente. head -10 pega apenas as 10 primeiras linhas.

Exercício 2 — Monitorando em tempo real

O **htop** é um monitor interativo que atualiza automaticamente a cada segundo, como um "gerenciador de tarefas" do Linux:

```
htop
```

Observe as colunas PID, CPU%, MEM% e TIME. Pressione q para sair.

Exercício 3 — Criando processos em paralelo

O **sleep 300** cria um processo que "dorme" por 300 segundos. O **&** faz ele rodar em segundo plano:

```
sleep 300 &  
sleep 300 &  
sleep 300 &
```

Agora liste os processos para ver os 3 que acabou de criar:

```
ps aux | grep sleep
```

O símbolo | (pipe) passa o resultado de um comando para outro. Aqui, filtra apenas os processos sleep.

Exercício 4 — Encerrando processos com sinais

No Linux, para encerrar um processo você envia um "sinal". Os dois mais comuns são:

- **SIGTERM** — pede educadamente para o processo encerrar.
- **SIGKILL** — força o encerramento imediatamente, sem chance de o processo reagir.

```
kill -SIGTERM <PID>  
kill -SIGKILL <PID>
```

Substitua <PID> pelo número que apareceu no ps aux. Com SIGTERM o processo pode ignorar o sinal, com SIGKILL não.

Exercício 5 — Simulando carga no sistema

O **stress-ng** força o processador a trabalhar intensamente. O **--cpu 2** usa 2 núcleos e **--timeout 30s** para após 30 segundos:

```
stress-ng --cpu 2 --timeout 30s &  
htop
```

Observe como o uso de CPU dispara. Compare com como estava antes de rodar o stress-ng.

Como encerrar o laboratório

Saia do container:

```
exit
```

Desligue o container:

```
docker compose down
```

O docker compose down para e remove o container. Tudo que foi feito dentro dele é apagado — o ambiente volta do zero na próxima vez que subir.